Лекция 1. Введение в машинное обучение и линейные модели

О интенсиве

В рамках данного интенсива мы последовательно познакомимся с основами классического машинного обучения (MO, ML) — от постановки задачи и устройства моделей до подходов к их обучению и оценки. Основной упор будет сделан на интуитивное понимание и математику, лежащую в основе моделей, а также на важные практические аспекты: работу с данными, выбор модели и т.д.

Примеров кода здесь **не будет** (или будут, но крайне ограниченно, скорее как fun-факты). В ходе интенсива иногда будут появляться Jupyter тетрадки, в которых можно ознакомиться с кодами.

Что такое машинное обучение?

Перед тем как начинать какие-то рассуждения, важно понимать, что такое машинное обучение и какие задачи ставит перед собой данная область. *Машинное обучение* (ML, MO) — наука, решающая задачи поиска закономерностей в ограниченных данных с помощью построения т.н. *модели, или алгоритма обучения*. В рамках наших занятий мы будем рассматривать раздел *обучения с учителем*.

Обучениие с учителем (supervised learning) представляет собой класс задач, в котором в рамках данных известна целевая переменная (target), которую алгоритм учится предсказывать. Пока непонятно, что такое «учится предсказывать», но к этому мы вернёмся ниже. Важна сама суть: пусть нам необходимо предсказать цену квартиры, основываясь на таких критериях, как площадь, этаж, район и т.д. Сначала нам дают всю необходимую информацию вместе с ценой, чтобы мы смогли изучить, какова реальная цена в зависимости от набора признаков. Затем нам дают другой набор данных, с теми же признаками, но уже без цены, и наша задача — *предсказать* эту стоимость. Целевая переменная (стоимость квартиры) выступает «учителем», по которому мы изучаем влияние данных признаков на стоимость. Вышеописанный пример относится к задаче *регрессии*.

Данные играют ключевую роль в машинном обучении. Именно на основе данных модель выявляет закономерности и формирует свою предсказательную способность. Здесь важно понимать, что мы не можем гарантированно знать, как и откуда появились наши данные, поэтому не стоит рассматривать машинное обучение как таблетку от всех проблем — *модели неизбежно допускают ошибки*, а насколько критичные, зачастую, зависит от качества данных (и ещё от нескольки критериев, которые слишком рано брать в голову). Поэтому важно не только построить алгоритм, но и тщательно понимать, с какими данными мы работаем, как они устроены и какие потенциальные сложности в себе несут. Об этом подробнее — в следующем разделе.

Основные понятия

Представьте себе обычную таблицу, где присутсвуют именованные столбцы — имена этих столбцов будем называть *признаками*.

Объектом будем называть единицу, для которой необходимо совершить предсказание, в контексте таблицы — одна строка. Возвращаясь к примеру с квартирами, объектом будет являться одна конкретная квартира.

Множество всех доступных объектов будем обозначать через $\mathbb{X}$, целевую переменную (и соответственно, множество всех целевых переменных) — через $y \in \mathbb{Y}$. Совокупность (x, y) называется *обучающим примером*, а их полное множество — *обучающей выборкой*:

$$X = (x_1, y_1), (x_2, y_2), \ldots, (x_\ell, y_\ell)\ X = (x_i, y_i)_{i=1}^{\ell},$$

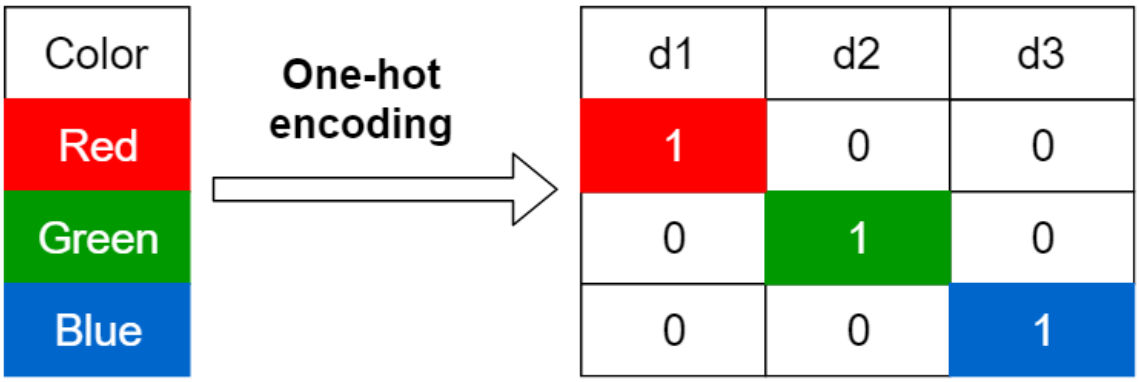
где ℓ — их количество.

Конечно, мы не можем напрямую дать компьютеру задачу «предскажи цену квартиры», так как компьютер не понимает, что такое квартира, площадь квартиры и так далее. Поэтому необходимо представить объект в виде набора признаков — т.н. *признаковое описание*. Объект $x \in \mathbb{X}$ можно представить вектором $(x_1, x_2, \ldots, x_d)$, где x_i — i-й признак данного объекта, а d — количество признаков. Такие признаки могут быть разными, среди основных выделяют:

- Числовые: $x_i \in \mathbb{R}$ (например, площадь квартиры, цена);
- Категориальные: $x_i \in \{c_1, c_2, \ldots, c_k\}$, где множество категорий неупорядочено (например, район города);
- Упорядоченные категориальные: категориальные с чётким порядком (например, оценка по 5-балльной шкале);
- Множествозначные (set-valued): значение x_i является подмножеством универсального множества (например, хэштеги под видео). Иными словами — когда значение признака можно задать с помощью нескольких категориальных признаков;
- Временные (time-based): признаки, связанные со временем, например, дата покупки, час обращения, год выпуска.

Как мы обсуждали ранее, мы хотим каждый объект представлять в векторной форме $(x_1, ..., x_d)$. Внимательный читатель мог задаться вопросом — а как мы собираемся помещать в вектор нечисловые признаки? Скажем, что если у нас есть категориальный признак в наборе данных?

В таких случаях можно (и нужно) прибегнуть к специальным кодирующим алгоритмам. Так, наиболее популярным алгоритмам для кодирования признаков является one-hot-encoding. Его идея заключается в следующем: пусть у нас есть признак $color$, который является категориальным и принимает значения из множества $\{A, B, C\}$. В результате работы алгоритма, вместо признака $color$ появляются признаки $colorA$, $colorB$,$colorC$, затем присваивается 1 или 0 в соответствии тому значению, которое было ранее у каждого объекта.



В зависимости от значений, которые принимает y, задачи машинного обучения также делятся на:

— **Регрессия:** $\mathbb{Y} = \mathbb{R}$ — предсказание числовых значений (например, цена квартиры);

— **Бинарная классификация:** $\mathbb{Y} = 0, 1$ — два возможных класса (например, болезнь есть/нет);

— **Многоклассовая классификация:** $\mathbb{Y} = 0, 1, \ldots, K$ — более двух категорий (например, прогноз погоды: «ясно», «облачно», «дождь»);

Линейные модели

Теперь, с этим набором знаний, наконец-то можно начать что-то творить. Сегодня мы остановимся на задаче регрессии ($\mathbb{Y} = \mathbb{R}$). Для понимания можно представить, что существует какая-то неизвестная нам функция, которая идеально справляется с задачей предсказания, т.е. идеально описывает наши данные. В таком ключе, наша задача сводится к построению функции, которая будет как можно ближе к вышеописанной:

$$a : \mathbb{X} \to \mathbb{Y}$$

Конечно, рассматривать все подряд функции бессмысленно и долго, поэтому обычно сначала фиксируют какое-то семейство функций, которое будет рассматриваться. Сегодня мы поговорим о семействе *линейных моделей*, которое можно записать как:

$$\mathcal{A} = \{a \mid a(x) = w_0 + w_1 x_1 + \cdots + w_d x_d\}\ ,\ w_i \in \mathbb{R}$$

т.е. все такие модели a, которые можно представить в виде линейной комбинации с... А что это?

Значения w называются *весами* — это такие параметры модели, через подбор которых модель научится выдавать предсказания. Значение w_0 выделяют отдельно и называют bias (смещение). Как осуществляется подбор этих параметров — пока что чёрный ящик, о котором сейчас не стоит задумываться. Лишь скажем, что процесс подбора этих w называется *обучением модели*.

Значит, наша модель берёт все признаки, каждый домножает на какой-то специально подбираемый коэффициент, суммирует все полученное, и на выходе получает величину, которая, хотелось бы, является предсказанием.

Мы знаем (а если не знаем, то теперь знаем), что линейную комбинацию можно записать в виде скалярного произведения векторов, которое в свою очередь выглядит так

$$a(x) = w_0 + \langle w, x \rangle$$

Чтобы записать это совсем красиво, нам мешает w_0. Внесём следующее предположение: первый признак будет единичным (это легко сделать, просто нужно добавить к данным единичный столбец). Тогда w_0 станет весом этого

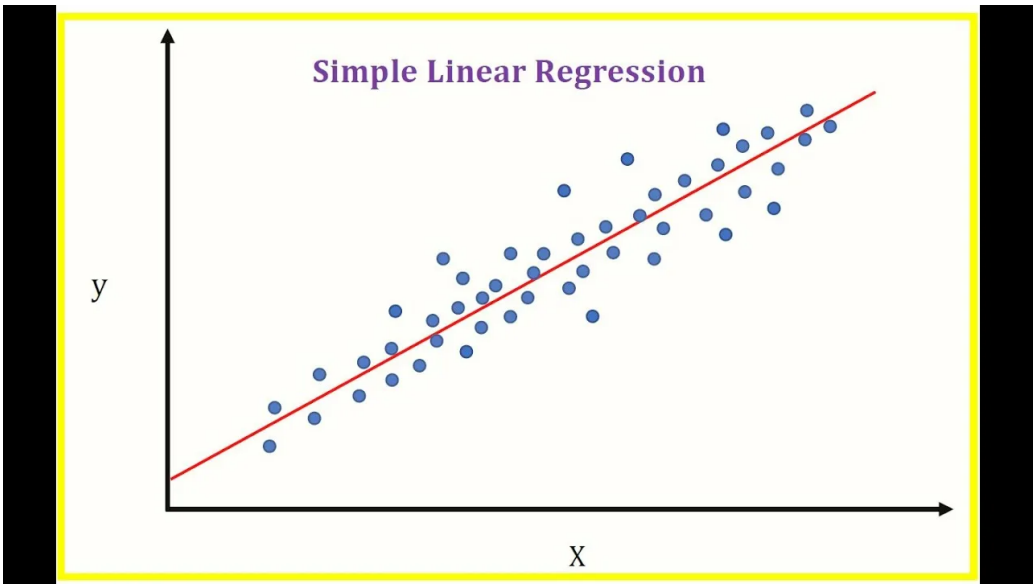
признака, а вся запись получит вид:

$$a(x) = \langle w, x \rangle$$

Только что мы вывели модель *линейной регрессии*. Если мы построим линейную регрессию для одного признака, то получим

$$a(x) = w_1 x_1 + w_0$$

Вам это точно ничего не напоминает? Всё верно — это самое простое уравнение прямой. Отсюда вытекает следующая идея: геометрически идея линейной регрессии заключается в том, чтобы построить прямую (если признаков больше, то гиперплоскость), которая наилучшим образом описывает данные. Фразу «наилучшим образом описывает данные» можно наглядно увидеть на изображении ниже.



Интуитивно то понятно, но как более формально определить наши наблюдения? Между каждой точкой и построенной прямой есть некоторое расстояние, и задача линейной регрессии в данном случае сводится к тому, чтобы построить прямую, у которой сумма всех таких расстояний наименьшая. Загвоздка лишь в том, что мы не можем заранее знать, когда эта сумма наименьшая.

Давайте также затронем ещё один момент, касающийся данных. Представим, что на изображении есть ещё одна точка, которая лежит очень далеко от всех остальных точек (и от прямой соответственно). Это означает, что этот объект содержит какое-то аномальное значение признака или даже нескольких

признаков (например, рост 200м). В общем, не суть — так или иначе у нас присутствует точка, которая в этом смысле сильно выделяется на фоне остальных. Такие объекты называются *выбросами* (outliers).

Важно сразу определить серьёзный недостаток данной модели — мы делаем очень сильное предположение о том, что целевую переменную можно выразить с помощью линейной комбинации признаков, т.е. мы предполагаем, что между признаками и целевой переменной существует *линейная зависимость*. На практике такое бывает крайне редко, но от этого ознакомление с этой моделью не становится бессмысленной — на её примере мы разобрали задачу регрессии, узнали про какие-то веса и к чему в общем смысле сводится задача регрессии.

Также мы вскользь упомянули, что модель подбирает эти самые параметры до тех пор, пока не построит хорошую прямую. Но в какой момент мы поймём, что построенная прямая — достаточно хороша? Почему мы не можем просто построить пряммую, которая будет идеально предсказывать имеющиеся цены? По какому принципу обновляются веса? Об этом всём вы узнаете на следующем занятии.